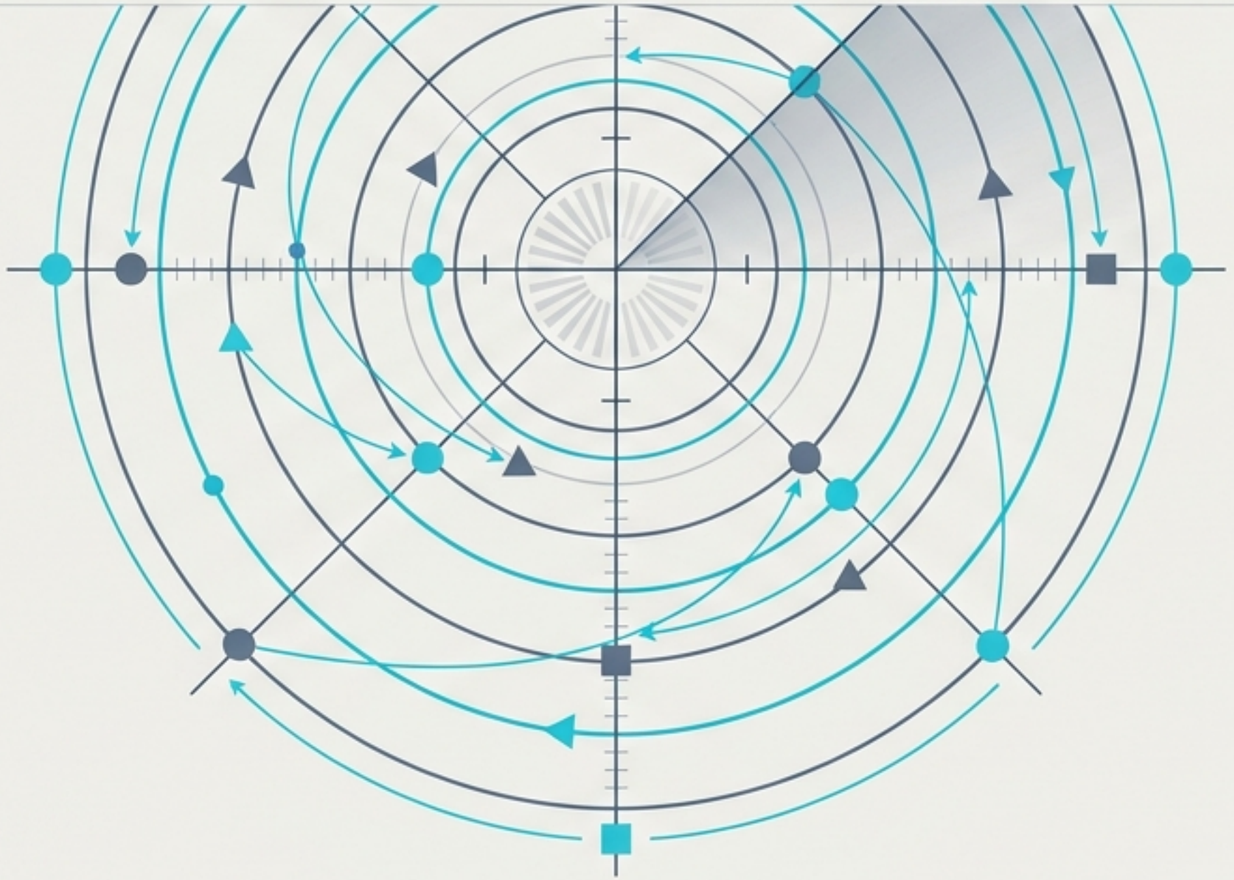
BEYOND FINDING BUGS

TESTING AS AN ARCHITECTURAL FEEDBACK LOOP



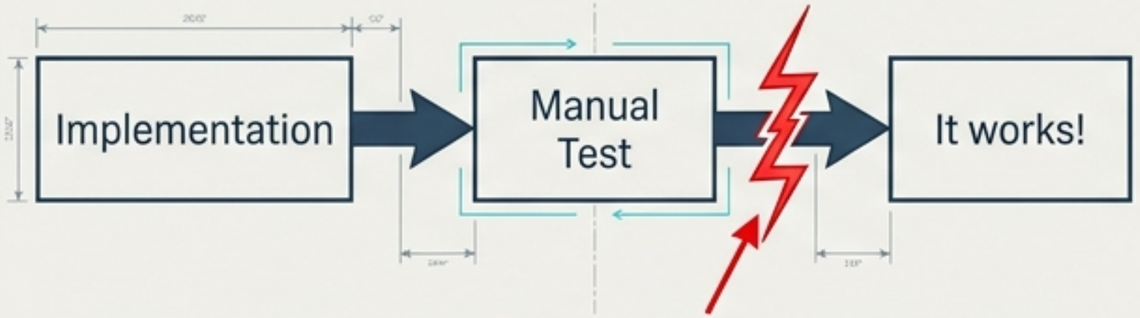
A paradigm shift for software engineers, tech leads, and architects.

Gemini Notebook

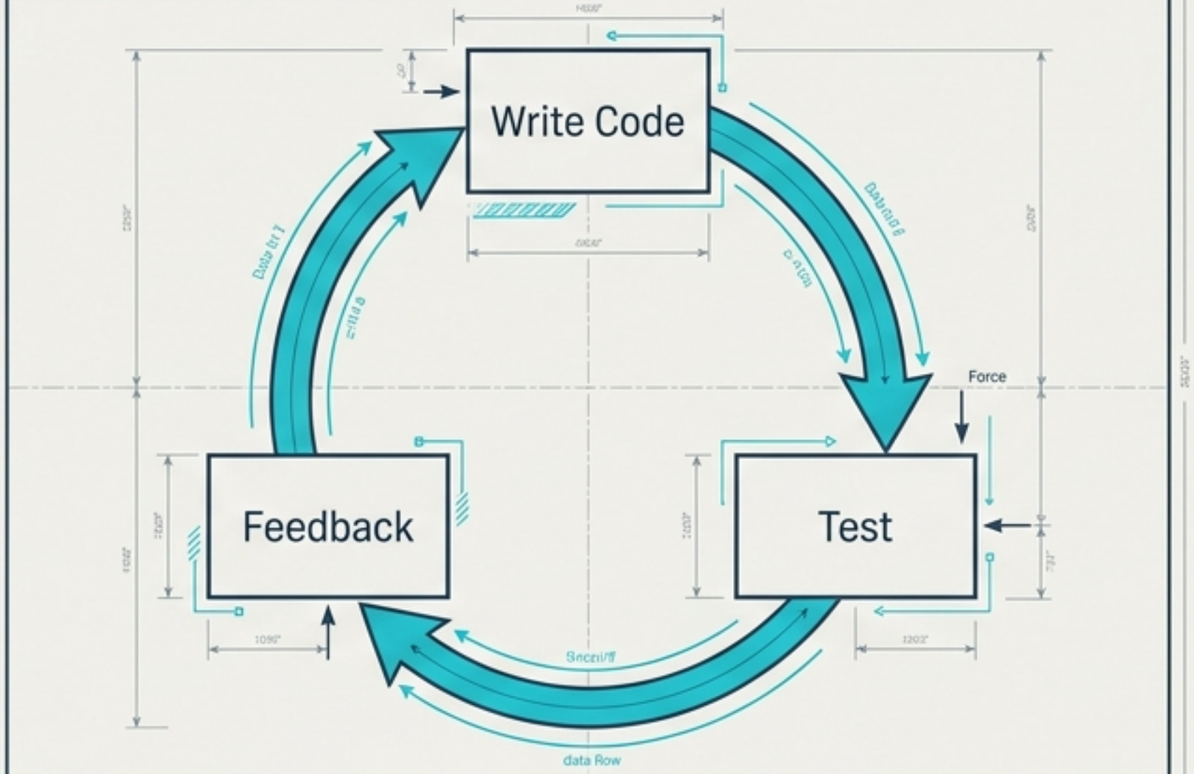
The Central Question of Software Engineering

How do we know our software still works while we constantly change it?

The Linear Trap



The Feedback Engine

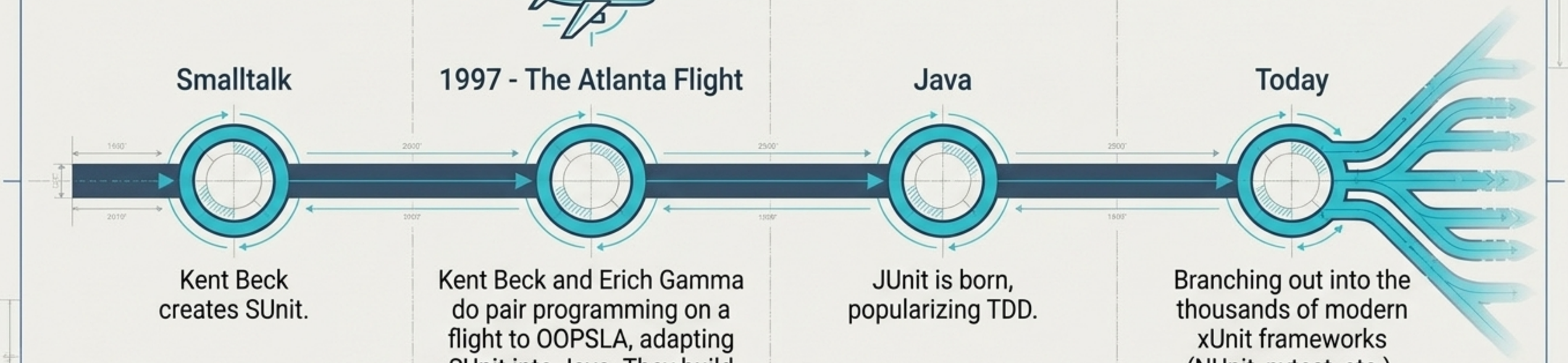


Software mutates constantly (new requirements, refactoring, architecture shifts).
The goal isn't just detecting bugs—it's obtaining rapid, reliable feedback on the impact of change.

Gemini Notebook

A Brief History of xUnit: Built for Design





Smalltalk

Kent Beck creates SUnit.

1997 - The Atlanta Flight

Kent Beck and Erich Gamma do pair programming on a flight to OOPSLA, adapting SUnit into Java. They build it test-first.

Java

JUnit is born, popularizing TDD.

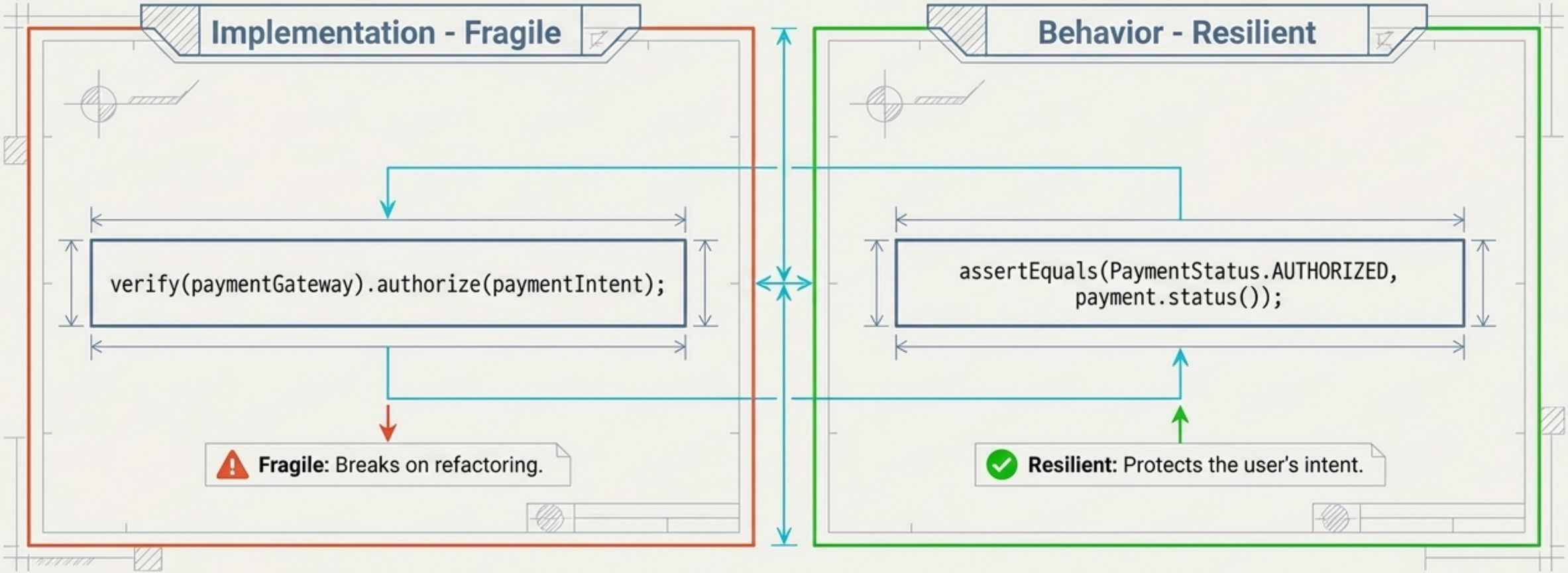
Today

Branching out into the thousands of modern xUnit frameworks (NUnit, pytest, etc.).

Automated testing wasn't an afterthought; it was engineered by the pioneers of modern software to enable continuous design.

Gemini Notebook

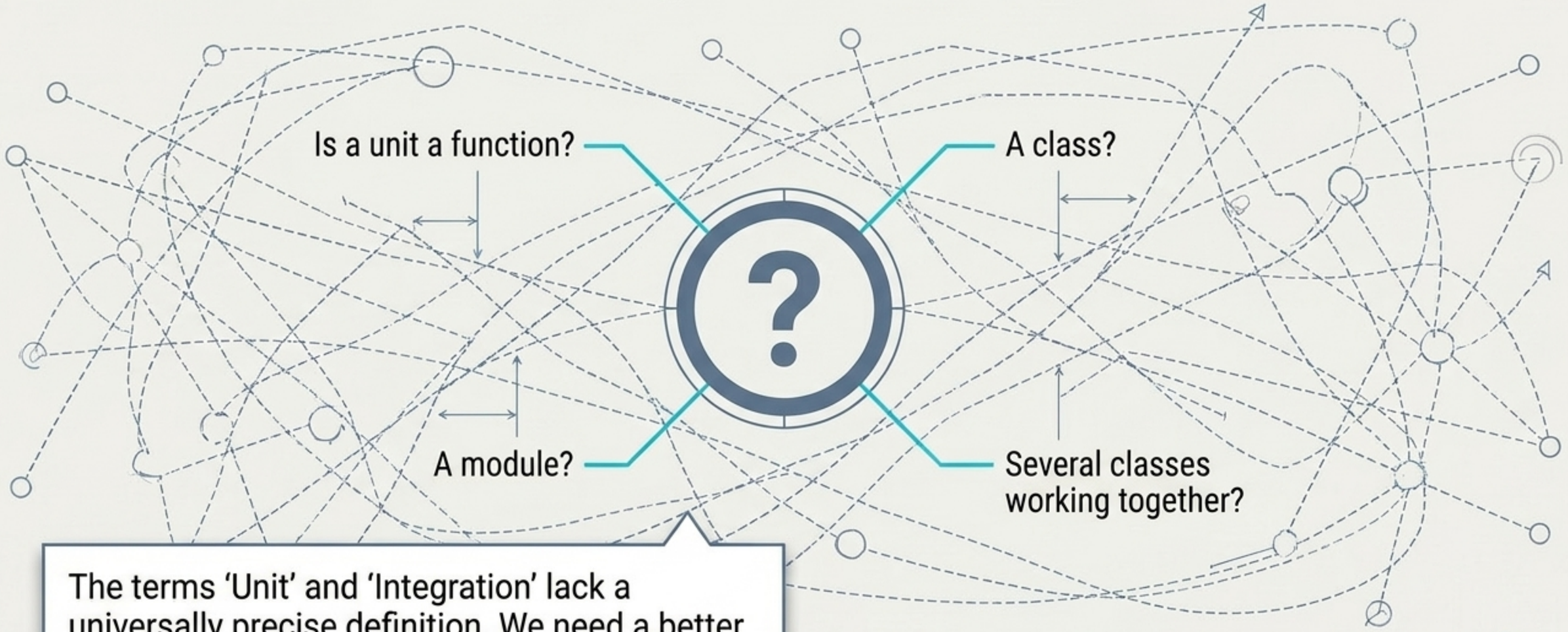
What Are We Actually Testing?



Test what the software does, not how it does it.
Observable behavior over implementation details.

Gemini Notebook

The ‘Unit vs. Integration’ Trap



The terms ‘Unit’ and ‘Integration’ lack a universally precise definition. We need a better taxonomy to guide our engineering decisions.

Ted Young’s Pragmatic Distinction

Gemini Notebook

A Better Mental Model: The I/O Taxonomy

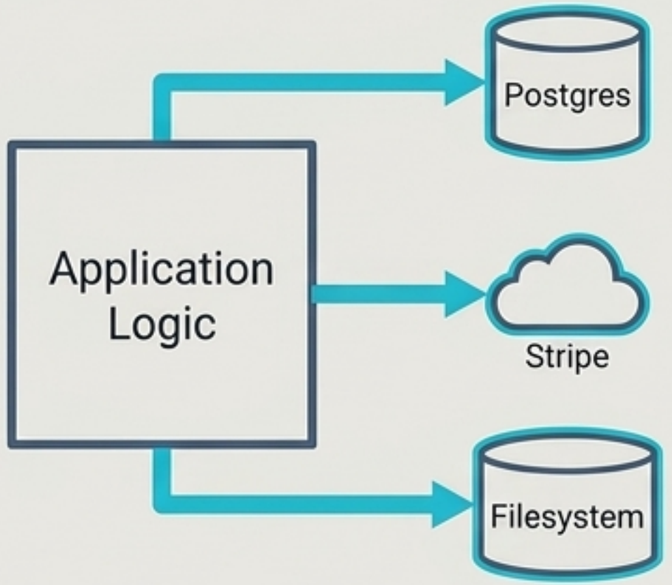
I/O-Free (The Core)

Application Logic

- ✓ No Database
- ✓ No Network
- ✓ No Filesystem
- ✓ No External Clock

Profile: Fast + Deterministic + Cheap

I/O-Dependent (The Boundary)



Interacts with the outside world.

Profile: Slower, Complex, Essential for real integration confidence.

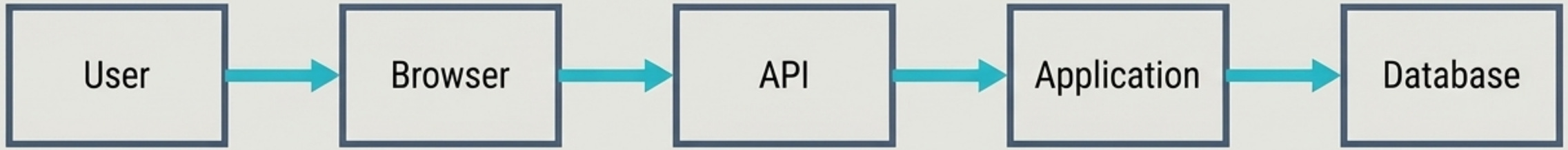
This distinction is far more useful than debating if a test is a 'Unit' or an 'Integration'.

Gemini Notebook

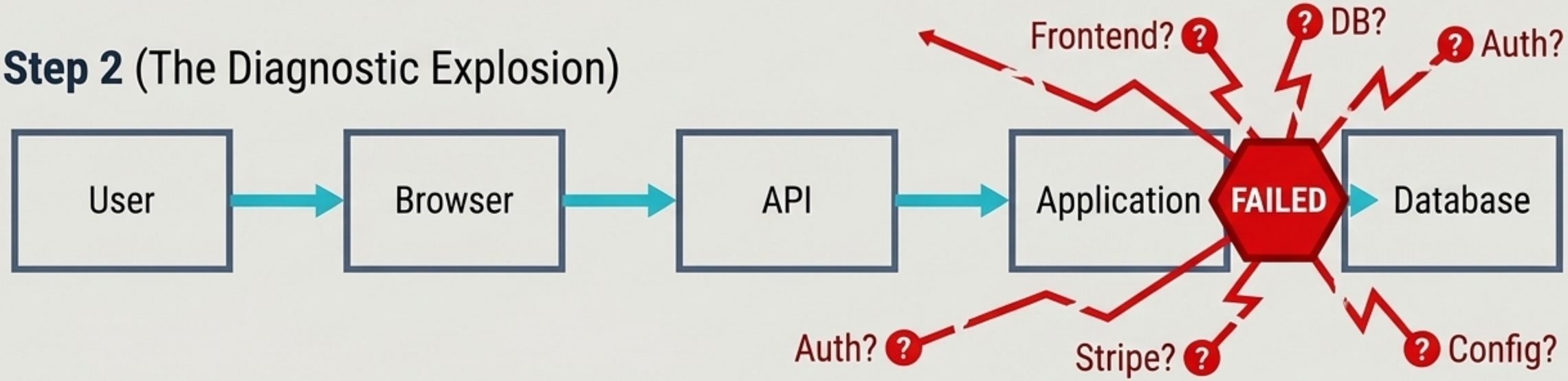
The E2E Diagnostic Trap

Step 1 (The Happy Path)

High Confidence: The whole system works!



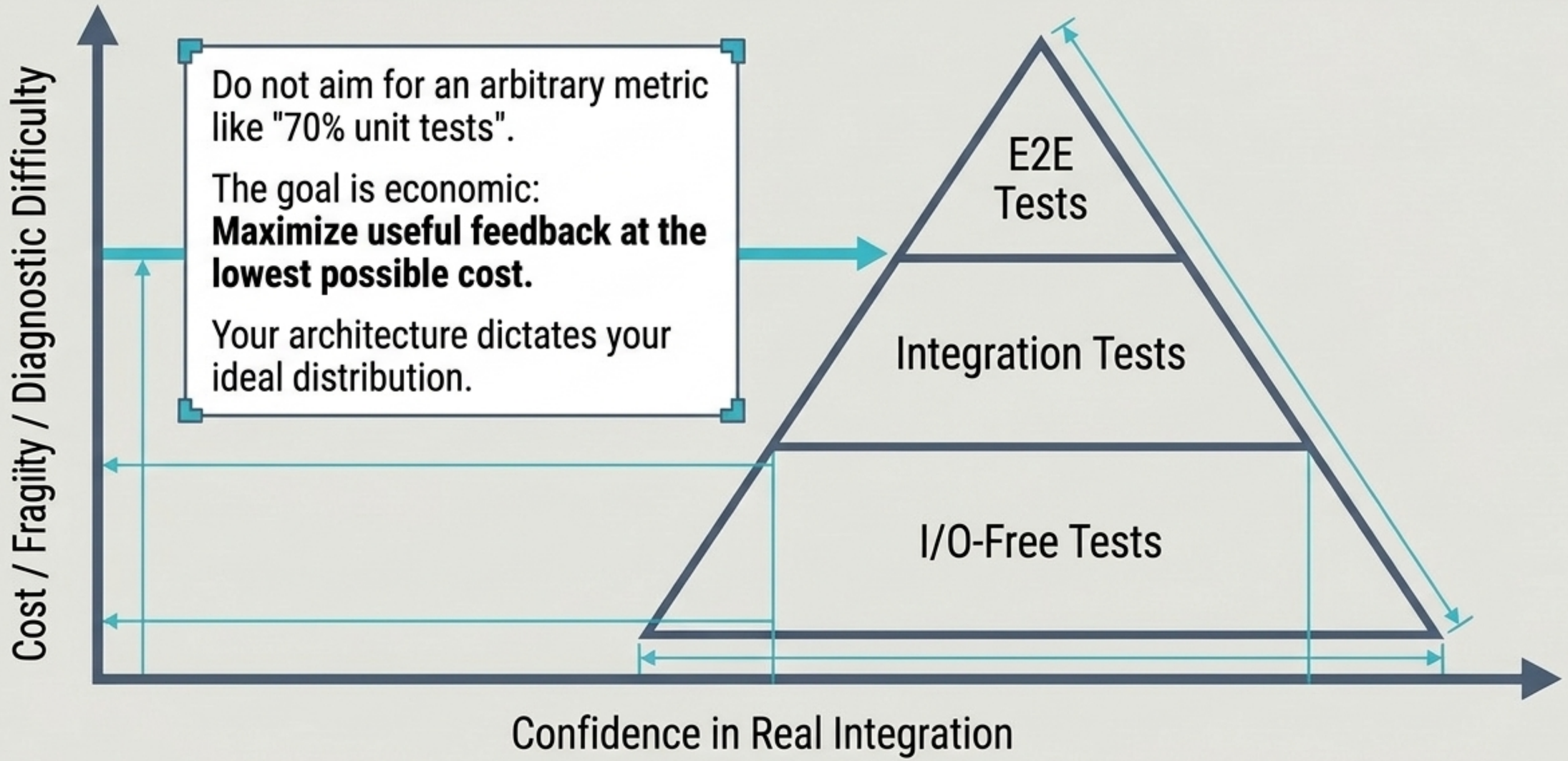
Step 2 (The Diagnostic Explosion)



End-to-End tests prove the system *can* work, but when they fail, they are terrible at telling you *why*.

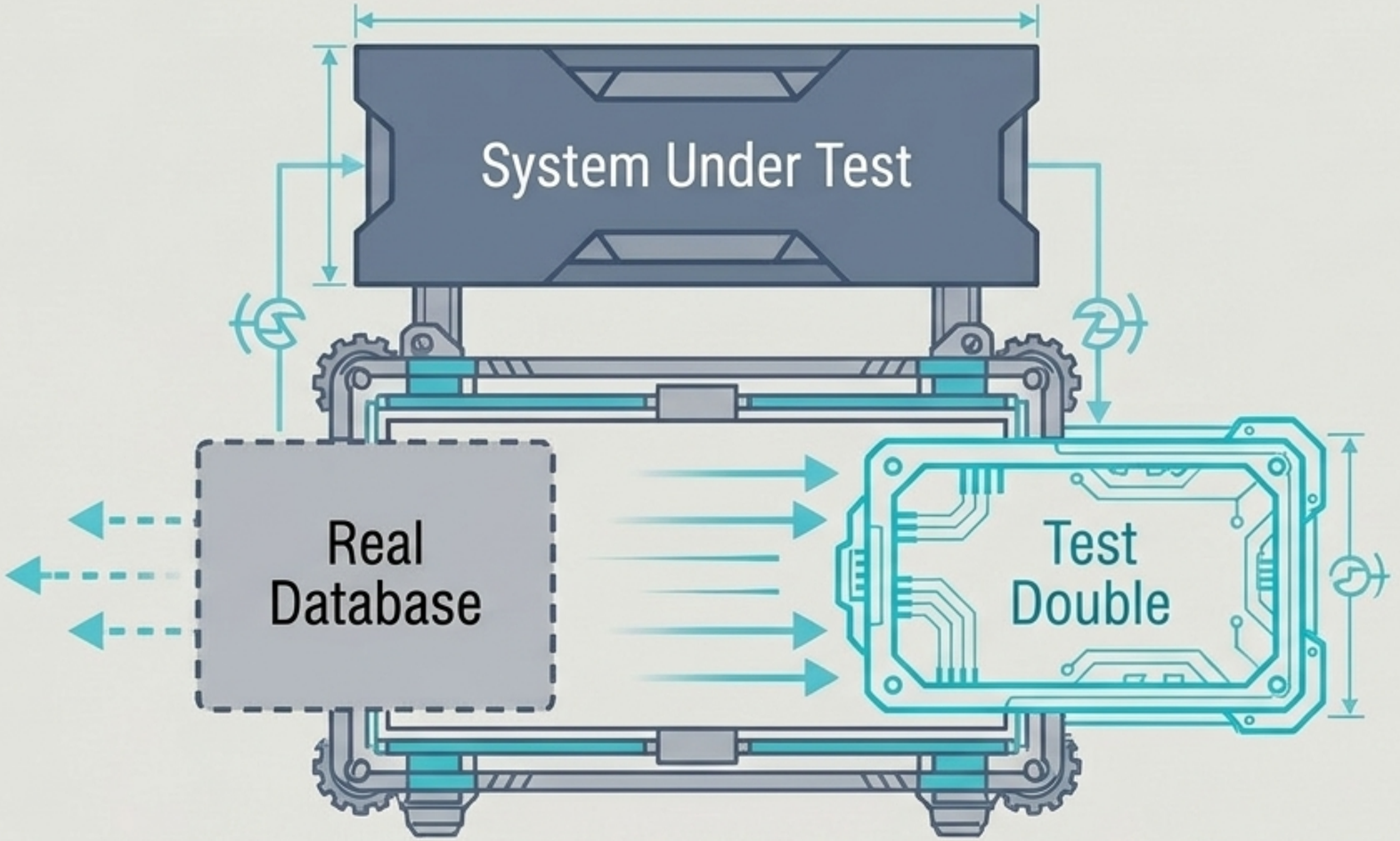
Gemini Notebook

The Test Pyramid is a Heuristic, Not a Law



Gemini Notebook

Managing Boundaries with Test Doubles



- Test Double
  - Dummy
  - Stub
  - Fake
  - Spy
  - Mock

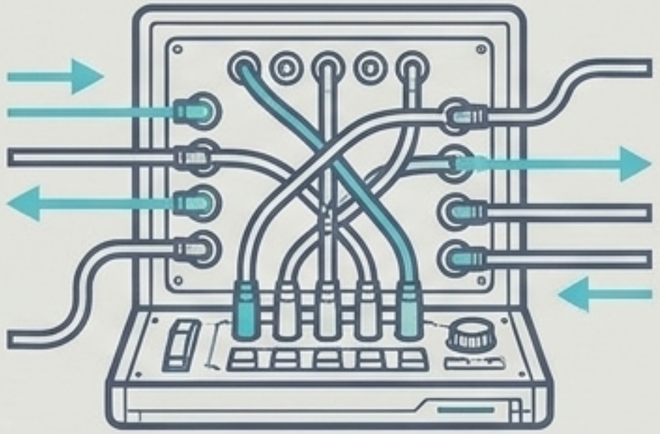
How do we test logic that depends on slow, external I/O? We swap the dependency.

Gemini Notebook

Mocks Aren't Stubs: Clarifying the Doubles

Stub

Focus: Input / State



```
when(repository.findById(id)).thenReturn(user);
```

Metaphor: A controlled switchboard.
“When asked X, answer Y.”

Mock

Focus: Output / Interaction



```
verify(emailService).send(email);
```

Metaphor: A security checkpoint camera.
“Did the system call this dependency correctly?”

Stubs control the response; Mocks verify the interaction.

Gemini Notebook

The Fragility Problem of Over-Mocking

Heavy Mocking

```
verify(repository.save());
verify(eventBus.publish());
```

Tests Pass

Harmless Refactoring

Code is changed to `repository.update()`. The actual system behavior remains 100% unchanged.

Fragile Failure

```
verify(repository.save()); // FAILS
```

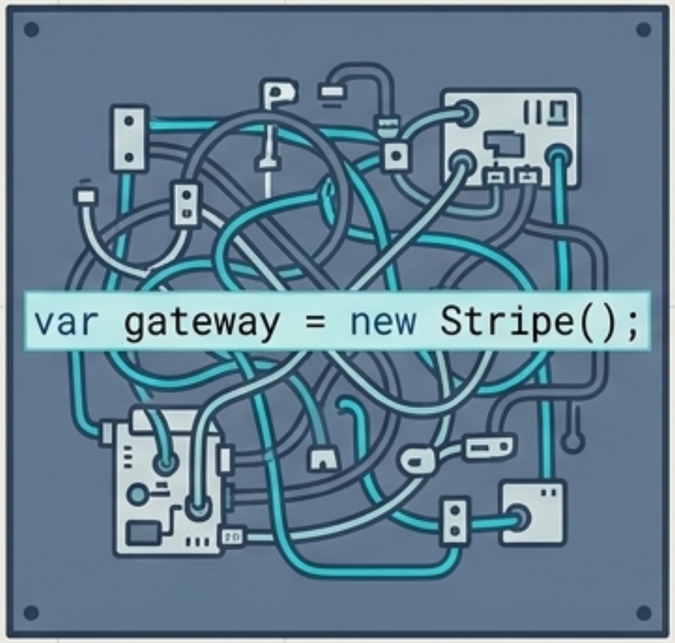
A test is fragile when changes that don’t affect behavior cause the test to break.

Tests should give you the freedom to refactor, not lock you into your current implementation implementation details.

Gemini Notebook

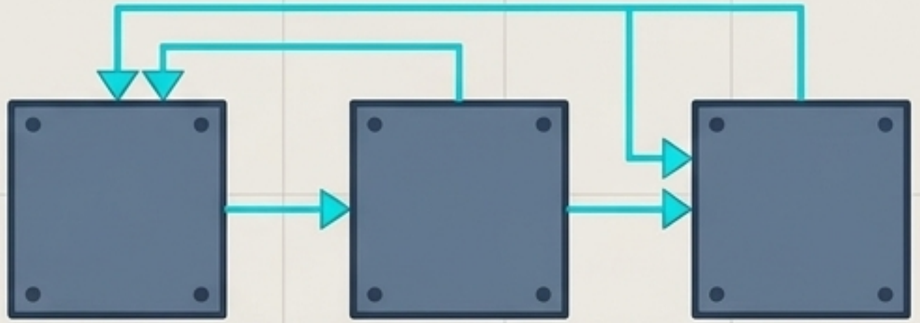
Testability is a Design Property

Pattern A (Hard to Test)



God Class with hidden dependencies.

Pattern B (Easy to Test)



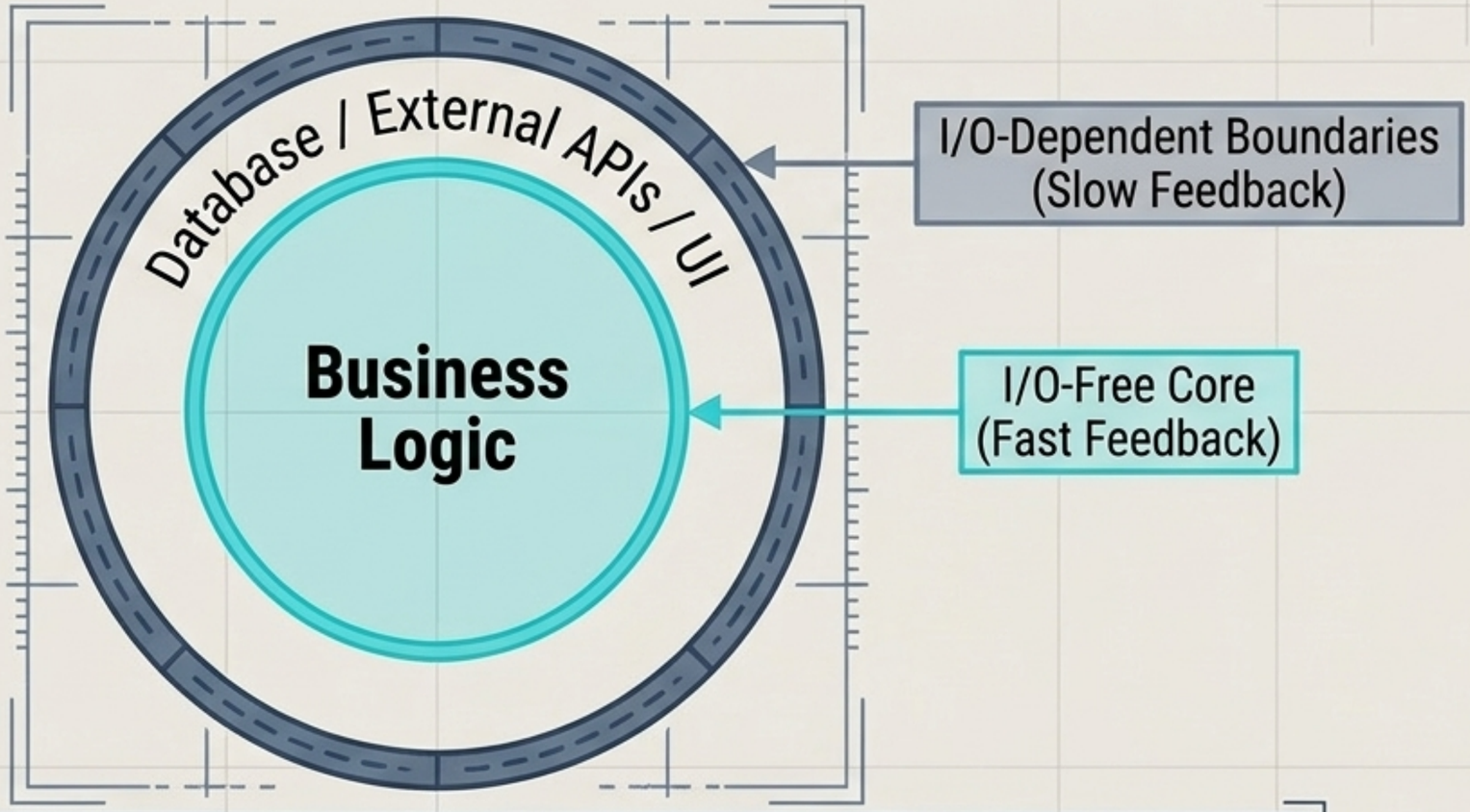
```
PaymentService(PaymentGateway gateway)
```

Clean, modular blocks with explicit dependencies.

If a codebase is hard to test, do not write a more complicated test. Ask: Why is this hard? It usually reveals too many responsibilities, hidden dependencies, or high coupling.

Gemini Notebook

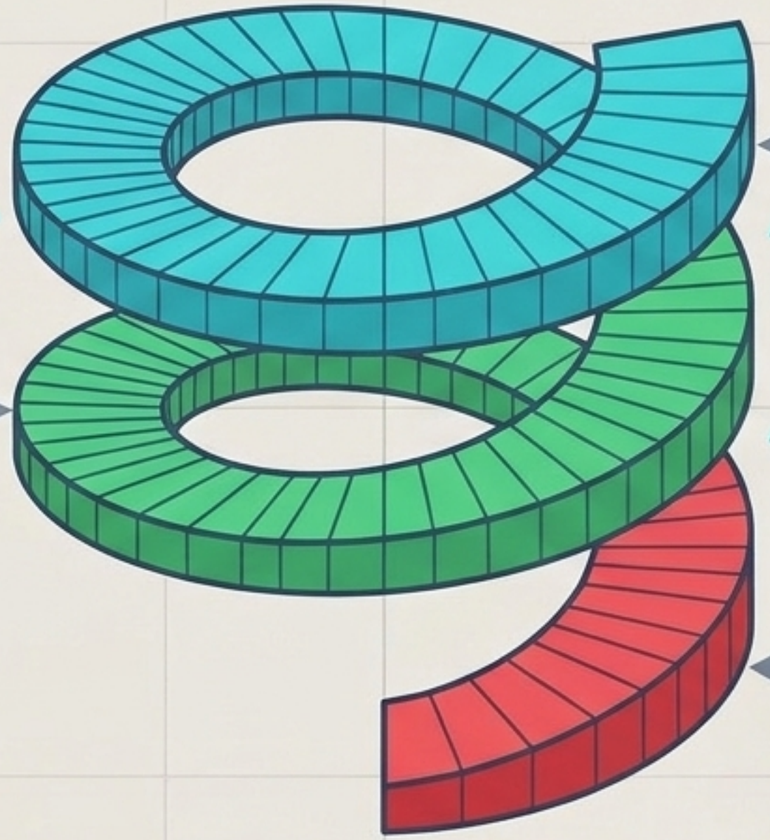
Clean Architecture Enables Test Isolation



By preventing core business logic from knowing about external technologies, we guarantee that our most important code can be tested entirely in the I/O-Free zone. Architecture dictates test speed.

Gemini Notebook

Demystifying TDD: The Design Engine



REFACTOR: Design Feedback

Improve structure, names, and coupling.

GREEN: Implementation

Make the test pass with minimal code.

RED: Intent

I know what I want the system to do, but it doesn't do it yet.

If writing the test is too hard, that is design feedback revealing poor architecture. TDD doesn't just test the design—it helps produce it.

Gemini Notebook

The 5 Core Tenets of Telemetry

1



Testing is Feedback.

It's about surviving change, not just finding bugs.

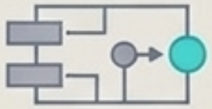
2



Fast Feedback Matters.

Distinguish between I/O-Free and I/O-Dependent tests.

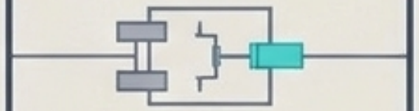
3



Test Behavior, Not Implementation.

Avoid the fragility of over-mocking.

4



Testability is Design.

Hard-to-test code is a symptom of architectural flaws.

5



TDD is a Loop.

Red-Green-Refactor is a continuous engine for design feedback.

Design for testability. Engineer for feedback.

Gemini Notebook